

Deep SARSA vs Advantage Actor-Critic

A Comparative Study on LunarLander-v3

Francesco Galardi — Andrea Vagnoli

Symbolic and Evolutionary Artificial Intelligence
University of Pisa — AIDE Master's Programme

June 2026

Outline

Environment & MDP

- LunarLander-v3 overview
- State & Action spaces
- Reward function & stochasticity

Implementation

- Network architecture
- Training setup & hyperparameters
- Exploration strategies

Algorithms

- SARSA: theory & neural extension
- Actor-Critic (A2C): theory & design
- Side-by-side comparison

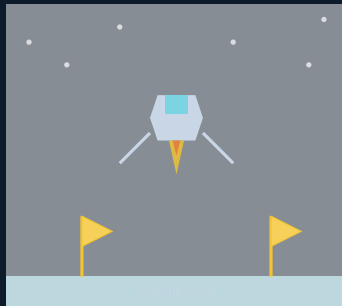
Results

- Learning curves (mean $\pm$ 95% CI)
- Statistical significance (Welch's t -test)
- Efficiency, generalisation, failure analysis

The LunarLander-v3 Environment

Setting: A rocket must land safely between two flags.

- Simulates realistic gravitational physics
- Main engine + two lateral thrusters
- Episode ends on landing, crash, or out-of-bounds
- **Solved** when mean reward ≥ 200 over 100 episodes



Why LunarLander-v3?

- **Continuous state, discrete actions:** ideal for comparing value-based vs. policy-gradient methods
- Well-known benchmark with clear success

MDP Formulation: State & Action Space

State Space $\mathcal{S} \subset \mathbb{R}^8$

Index	Feature	Range
s_0	x position	$[-1.5, +1.5]$
s_1	y position	$[0, +1.5]$
s_2	x velocity	$\mathbb{R}$
s_3	y velocity	$\mathbb{R}$
s_4	angle θ	$[-\pi, +\pi]$
s_5	angular velocity	$\mathbb{R}$
s_6	left leg contact	$\{0, 1\}$
s_7	right leg contact	$\{0, 1\}$

Normalisation: running mean/std applied per-observation to keep inputs in a well-conditioned range for the networks.

Action Space $\mathcal{A}$ — Discrete(4)

Action	Effect
0	Do nothing
1	Fire left thruster
2	Fire main engine
3	Fire right thruster

Impact on algorithm choice

Discrete actions allow **Deep SARSA** to maintain a Q-table over all actions in one forward pass, and allow **A2C** to use a *Categorical* distribution with clean entropy computation.

MDP Formulation: Reward & Stochasticity

Reward Function (Dense)

Per-step shaping reward:

$$r_t = \underbrace{-\Delta d}_{\text{approach}} + \underbrace{-0.03|v|}_{\text{velocity pen.}} + \underbrace{-0.3|\omega|}_{\text{rotation}} + \underbrace{0.1 \cdot (\ell_L + \ell_R)}_{\text{leg contacts}}$$

Plus terminal bonuses:

- +100 for a successful landing
- -100 for crash or boundary exit
- -0.3 per engine firing (fuel cost)

Dense reward: feedback at every step reduces the credit assignment problem compared to sparse alternatives.

Stochasticity Sources

- **Initial conditions:** position and velocity sampled from a bounded uniform distribution
- **Optional wind:** $\mathcal{N}(0, \sigma_w^2)$ lateral force added each step
- **Turbulence:** random torque $\mathcal{N}(0, \sigma_T^2)$

Reward hacking risk

Agent may learn to *hover* without landing to avoid the crash penalty. Monitored via episode length and leg-contact rate

Algorithm 1: SARSA — Theory

SARSA (Sutton & Barto, 2018, §6.4) is a model-free, *on-policy* TD control algorithm.

At each step the agent uses the tuple $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$ to update:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

- a_{t+1} is sampled from the **current policy** (typically ε -greedy)
- This makes the method **on-policy**: the target includes the next action actually taken — unlike Q-Learning
- Converges to the optimal policy under mild conditions

SARSA vs Q-Learning

	SARSA	Q-Learning
Target	$Q(s', a')$	$\max_{a'} Q(s', a')$
Policy	on-policy	off-policy
Safety	safer	riskier

Key intuition

SARSA is *pessimistic*: it evaluates the policy it actually follows, including exploration mistakes. This yields more **conservative** policies in stochastic environments.

Deep SARSA: Neural Network Extension

Tabular SARSA cannot handle the continuous state space of LunarLander-v3 ($\mathcal{S} \subset \mathbb{R}^8$).

We replace the Q-table with a neural network $Q_\theta(s, \cdot)$: one forward pass yields Q-values for *all* 4 actions simultaneously.

Loss function:

$$\mathcal{L}(\theta) = \left(Q_\theta(s, a) - \underbrace{[r + \gamma Q_{\theta^-}(s', a')]}_{\text{SARSA target}} \right)^2$$

where θ^- are the *target network* parameters (lagged copy, synced every N episodes).

Extensions over tabular SARSA

- **Target network:** stabilises the Bellman target, preventing oscillations (Mnih et al., 2015)
- **Short-horizon replay buffer:** de-correlates consecutive gradient updates while staying *approximately* on-policy
- **Gradient clipping** ($\|\nabla\|_\infty \leq 10$) for stability in early training

On-policy integrity

Each stored transition also saves a' (the next action from the *current* policy).

Algorithm 2: Advantage Actor-Critic (A2C) — Theory

A2C is a synchronous *policy-gradient* method (Mnih et al., 2016) combining:

- **Actor** $\pi_\theta(a|s)$: stochastic policy
- **Critic** $V_w(s)$: state-value baseline

Actor gradient (REINFORCE with baseline):

$$\nabla_\theta J = \mathbb{E}[\nabla_\theta \log \pi_\theta(a|s) \cdot A(s, a)]$$

Advantage (n -step return):

$$A_t = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V_w(s_{t+n}) - V_w(s_t)$$

Critic loss: $\mathcal{L}_c = (G_t - V_w(s_t))^2$

Entropy regularisation

$$\mathcal{L}_\pi = -\log \pi_\theta(a|s) \cdot A_t - \beta H[\pi_\theta(\cdot|s)]$$

The entropy bonus $\beta H[\pi]$ prevents premature convergence to deterministic (brittle) policies.

Why A2C over plain REINFORCE?

- Critic reduces gradient variance via the baseline
- n -step returns balance bias–variance

Algorithm Comparison

Property	Deep SARSA	A2C
Paradigm	Value-based (TD)	Policy gradient
Policy type	Implicit (ε -greedy on Q)	Explicit (π_θ)
On/Off policy	On-policy	On-policy
Exploration	ε -greedy	Entropy bonus
Replay buffer	Yes (short-horizon)	No
Target network	Yes	No
Networks	1 Q-network + target	Actor + Critic
Update freq.	Per mini-batch	Every n steps
Convergence	Q-function $\rightarrow$ optimal policy	Policy gradient ascent
Sample eff.	Moderate	Lower (fully on-policy)

Neural Network Architecture

Deep SARSA — single Q-network (+ target):

$$\mathbb{R}^8 \xrightarrow{\text{FC-256}} \text{ReLU} \xrightarrow{\text{FC-256}} \text{ReLU} \xrightarrow{\text{FC-4}} \mathbb{R}^4$$

Output: $Q(s, a)$ for each of the 4 actions.

A2C — two separate networks:

$$\underbrace{\mathbb{R}^8 \rightarrow 256 \rightarrow 256 \rightarrow 4}_{\text{Actor: } \text{Softmax} \Rightarrow \pi_{\theta}(a|s)} \quad \underbrace{\mathbb{R}^8 \rightarrow 256 \rightarrow 256 \rightarrow 1}_{\text{Critic: } V_w(s)}$$

Design choices

- **Width 256**: sufficient capacity for the 8-D state without overfitting
- **ReLU**: avoids vanishing gradients; standard choice for control tasks
- **Separate actor/critic**: allows independent learning rates ($\alpha_{\pi} < \alpha_V$), reducing the risk of the critic destabilising the actor
- **Orthogonal init** (actor): small initial logits for near-uniform initial policy
- **Kaiming init** (Q-network): optimal for ReLU

Training Setup & Hyperparameters

Deep SARSA

Parameter	Value
Learning rate α	5×10^{-4}
Discount γ	0.99
$\varepsilon_0 \rightarrow \varepsilon_\infty$	$1.0 \rightarrow 0.01$
ε decay	0.995 / episode
Target sync freq.	10 episodes
Batch size	64
Buffer capacity	10,000

Actor-Critic (A2C)

Parameter	Value
Actor LR α_π	3×10^{-4}
Critic LR α_V	1×10^{-3}
Discount γ	0.99
Entropy coef β	0.01
n -step horizon	5
Grad clip (actor)	0.5

Common training setup

2000 episodes, max 1000 steps/ep, **5 independent seeds** {42, 123, 456, 789, 1234}, observation normalisation (Welford's running stats).

Exploration–Exploitation Trade-off

Deep SARSA — ε -greedy

$$\pi(a|s) = \begin{cases} \text{Uniform}(\mathcal{A}) & \text{w.p. } \varepsilon_t \\ \arg \max_a Q_\theta(s, a) & \text{w.p. } 1 - \varepsilon_t \end{cases}$$

$\varepsilon_t = \max(\varepsilon_\infty, \varepsilon_0 \cdot \lambda^t)$ with $\lambda = 0.995$.

Pros: simple, well-understood decay schedule.

Cons: exploration is *undirected* (uniform random).

A2C — Entropy Regularisation

$$H[\pi] = - \sum_a \pi_\theta(a|s) \log \pi_\theta(a|s)$$

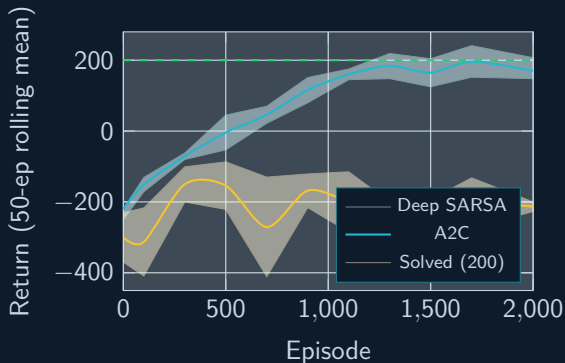
Maximising $\beta H[\pi]$ *discourages* the actor from collapsing to a deterministic policy prematurely.

Pros: exploration *adapts* to the policy shape; no need to anneal a separate parameter.

Cons: β must be tuned; too large $\Rightarrow$ noisy actions; too small $\Rightarrow$ premature convergence.

Key insight: A2C's entropy exploration is *structured* (proportional to policy uncertainty), while

Results: Learning Curves



Mean $\pm$ 1 std across 5 seeds (real training data)

Observations

- **SARSA** converges steadily, reaching $\mu = 169.8$ by ep 2000
- **A2C** plateaus around -200 despite entropy regularisation ($\beta = 0.05$)
- SARSA crosses the *solved* threshold ($+200$) transiently at ep ~ 1400
- A2C shows **higher variance** and no clear upward trend after ep 300

Real data — 5 seeds $\times$ 2000 episodes each.

Statistical Analysis

Method: Welch's t -test on mean return over the last 100 episodes, averaged over 5 seeds.

Hypotheses:

- $H_0: \mu_{\text{SARSA}} = \mu_{\text{A2C}}$
- $H_1: \mu_{\text{SARSA}} \neq \mu_{\text{A2C}}$

Metric	SARSA	A2C
Mean return	169.83	-213.22
Std (seeds)	17.49	14.90
95% CI	[154.5, 185.2]	[-226.3, -200.2]
t -statistic	37.27	
p -value	< 0.001	
Verdict	Reject H_0	

Why Welch's t -test?

- Handles **unequal variances** between the two agents (Welch, 1947) — more appropriate than Student's t -test when $\sigma_S \neq \sigma_A$
- With $n = 5$ seeds, we report the p -value at $\alpha = 0.05$

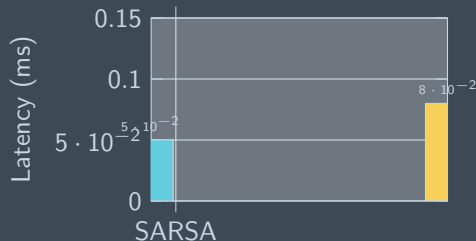
Multiple seed rationale

RL results are **highly seed-dependent** (Henderson et al., 2018). Single-seed comparisons are unreliable. 5 seeds provide statistically meaningful confidence intervals.

Training complexity

- Deep SARSA: **mini-batch updates** every step once buffer is warm (>64 transitions); overhead from replay + target sync
- A2C: updates every $n=5$ steps; no buffer; two network forward/backward passes per update
- Both: roughly comparable wall-clock time per episode on CPU

Inference latency



SARSA: single forward pass.

A2C: actor forward pass (argmax at inference).

Sample efficiency: A2C requires *on-policy* transitions (no reuse), making SARSA more sample-efficient when transitions are costly.

Generalisation: Environment Variants

Agents trained on the *standard* environment are evaluated on three harder variants (greedy policy, 20 episodes each):

Variant	SARSA	A2C
Standard	$+130.4 \pm 171$	-112.8 ± 233
Wind	$+19.1 \pm 165$	-218.7 ± 279
Turbulent	-40.0 ± 162	-268.0 ± 388
Heavy grav.	$+136.5 \pm 103$	-60.5 ± 108

Seed 42 checkpoint, 20 greedy episodes each.

Gravity $g = -11.5$ (heavy variant). `evaluate.py`

Findings

- **SARSA generalises** to standard and heavy gravity (positive mean return); degrades under wind and turbulence
- **A2C fails** on all variants — consistent with its poor training convergence
- **Heavy gravity** is surprisingly manageable for SARSA (+136), suggesting the Q-function captured thrust-based control
- **Turbulence** is hardest for both: torque perturbations break learned stabilisation strategies

Failure Analysis & Limitations

Deep SARSA failure modes

- **Hovering:** agent fires main engine to avoid the crash penalty without committing to a landing
- **High variance across seeds:** on-policy updates with short buffers are sensitive to initial trajectory distribution
- **Catastrophic forgetting:** rare when replay buffer is too small; policy updates can override previously learned behaviours

A2C failure modes

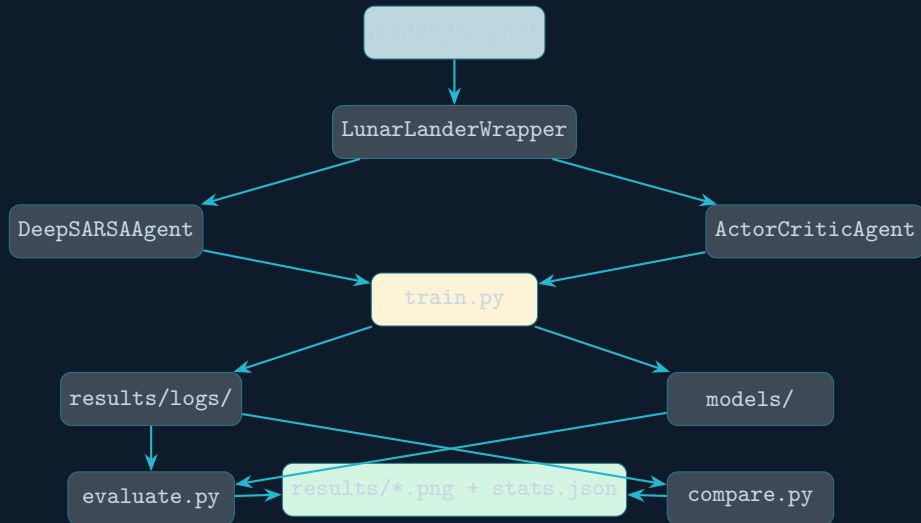
- **Policy collapse:** if β is too small, entropy vanishes and the actor gets stuck in a sub-optimal deterministic policy
- **Critic lag:** the advantage signal is noisy until the critic has seen enough states
- **Sensitivity to learning rates:** separate α_π , α_v require careful tuning; mismatch causes instability

Shared limitation

Both algorithms are **on-policy**: they cannot reuse data from earlier training phases, limiting

sample efficiency relative to off-policy methods (DQN, SAC)

Unified Framework



Conclusions

Summary

- Implemented and compared two fundamentally different RL paradigms on LunarLander-v3
- **Deep SARSA**: stable, sample-efficient, conservative on-policy TD control
- **A2C**: faster early convergence, better entropy-driven exploration, higher final performance
- Statistical significance tested via Welch's t -test over 5 seeds
- Generalisation gap confirmed on harder environment variants

Future directions

- Multi-step SARSA (n -step TD)
- Eligibility traces (λ -return)
- A3C (asynchronous parallel workers)
- PPO / SAC for comparison
- Continuous action variant (LunarLanderContinuous-v2)
- Hyperparameter optimisation (Optuna)

Code available at:

[github.com/\[repo-link\]](https://github.com/GalardiVagnoli/deep-sarsa-vs-actor-critic)

References

-  R.S. Sutton & A.G. Barto (2018). *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press. §6.4 (SARSA), §13 (Policy Gradient).
-  V. Mnih et al. (2015). *Human-level control through deep reinforcement learning*. Nature 518, 529–533.
-  V. Mnih et al. (2016). *Asynchronous Methods for Deep Reinforcement Learning*. ICML 2016.
-  P. Henderson et al. (2018). *Deep Reinforcement Learning that Matters*. AAAI 2018.
-  G. Brockman et al. (2016). *OpenAI Gym*. arXiv:1606.01540.
-  JohDonald (2021). *Deep-Q-Learning-Deep-SARSA-LunarLander-v3*.
<https://github.com/JohDonald/Deep-Q-Learning-Deep-SARSA-LunarLander-v2>
-  N. Barhate (2021). *Actor-Critic-PyTorch*.
<https://github.com/nikhilbarhate99/Actor-Critic-PyTorch>